

INFORME TECNICO ACTUALIZADO - SIMULADORSO

Estado de implementacion, pruebas, modulos, estructuras y requisitos

Proyecto: Simulador de planificacion de procesos y gestion de memoria Repositorio: /Users/vkrls/Documents/Proyectos/SimuladorSO Fecha de revision: 21 de junio de 2026 Version del informe: 2.0

Documento historico usado como referencia:

"Requerimientos Trabajo.pdf" Gestion de Procesos y Memoria, 13 diapositivas.

Nota:

El documento de requerimientos no esta actualizado. Se utilizo como una referencia comparativa, no como una especificacion contractual definitiva.

1. LEYENDA DE ESTADOS

[OK - PROBADO]

La funcionalidad esta implementada, compila y fue verificada mediante una prueba funcional o de integracion.

[OK - CODIGO]

La funcionalidad esta implementada y participa en el flujo actual. No se ejecuto una prueba especializada independiente para cada caso extremo.

[PARCIAL]

Existe una implementacion util, pero faltan comportamientos exigidos, precision, validaciones o integracion completa.

[PENDIENTE]

La estructura puede existir, pero la funcionalidad no esta implementada.

[DEUDA TECNICA]

No impide el flujo principal, pero debe corregirse para mejorar robustez, mantenimiento, precision o presentacion.

2. RESUMEN EJECUTIVO

El proyecto cuenta actualmente con un nucleo funcional en C y una interfaz grafica en Python/PySide6 conectada en tiempo real mediante stdin/stdout.

Estado global por area:

[OK - PROBADO] Planificacion de CPU:

- FCFS.

- SJF no apropiativo. - SJF apropiativo. - Round Robin.

- Prioridades no apropiativo. - Prioridades apropiativo.

[OK - PROBADO] Gestion de memoria:

- First Fit.

- Best Fit.

- Worst Fit.

- Lista enlazada de bloques. - Liberacion y fusion de huecos contiguos. - Direcciones base y limite almacenadas en el PCB. - Desperdicio interno calculado por proceso.

[OK - PROBADO] Cambio de contexto:

- Contexto activo dentro de Simulator.cpu_ctx. - Contexto guardado dentro de Pcb.cpu_ctx. - Guardado CPU -> PCB.

- Restauracion PCB -> CPU.

- Conservacion durante apropiacion, quantum, bloqueo y terminacion.

[OK - PROBADO] Entrada/salida basica:

- Teclado, disco e impresora.

- RUNNING -> BLOCKED.

- Espera dentro de io_q. - BLOCKED -> READY al concluir la operacion. - La simulacion no termina mientras existan procesos en io_q.

[OK - PROBADO] Integracion GUI/C: - Comandos CONFIG, ADD, RANDOM, SPEED, RUN, PAUSE y STOP. - Recepcion de snapshots JSON. - Visualizacion de PCB, colas, memoria, Gantt y estadisticas. - Cambio dinamico de First/Best/Worst Fit.

- Insercion de procesos mientras la simulacion esta ejecutandose.

[PARCIAL] Interrupciones: - Existen enum y campos dentro del PCB. - Se calcula una cantidad aleatoria entre 5 y 20. - No se generan, encolan ni atienden interrupciones reales.

[PARCIAL] Errores:

- Existe ERROR_STATE y ErrorData. - Memoria invalida produce ERR_MEM y termina en finished_q. - El error aleatorio de 0.5% se marca, pero no se ejecuta ni cancela.

[PARCIAL] Estadisticas:

- Espera, retorno, respuesta, throughput y utilizacion son mostrados. - El Gantt registra procesos, CPU inactiva y cambios de contexto sin perder el ultimo tick. - El tiempo de espera incluye indirectamente tiempo de E/S.

[PENDIENTE] Funcionalidad avanzada:

- Sistema real de interrupciones y señales. - Planificador de mediano plazo y estados suspendidos. - Stack/heap y asignacion variable durante ejecucion. - Colas independientes por dispositivo. - Comparacion automatizada de 20 procesos por politica. - Procesos internos del sistema operativo diferenciados de usuarios.

3. RESULTADOS DE VERIFICACION

3.1 Compilacion

[OK - PROBADO] C:
cc -std=c11 -Wall -Wextra -Wpedantic -Wconversion -Wshadow
Resultado:
src/main.c compila sin errores ni advertencias con esas opciones.
[OK - PROBADO] Python:
python -m compileall src
Resultado:
Todos los modulos Python pasan la validacion de sintaxis.

3.2 Matriz de planificacion y memoria
Se compilo src/main.c en un ejecutable temporal y se probaron tres procesos en cada combinacion posible:
FIRST FIT BEST FIT WORST FIT
FCFS OK OK OK
SJF no apropiativo OK OK OK SJF apropiativo OK OK OK Round Robin OK OK OK
Prioridad no apropiativa OK OK OK Prioridad apropiativa OK OK OK
Resultado:
18 de 18 combinaciones finalizaron los tres procesos con codigo de salida 0.

3.3 Prueba de entrada/salida
Se generaron 10 procesos aleatorios:
- Se observaron procesos en estado BLOCKED. - io_q alcanzo dos procesos simultaneos.
- Los procesos retornaron a ready_q. - Los 10 procesos terminaron. - El ejecutable finalizo con codigo 0.
Resultado:
El ciclo basico RUNNING -> BLOCKED -> READY -> RUNNING funciona.

3.4 Cambio dinamico de politica de memoria
Con el subproceso C activo se cambio la politica a Worst Fit.
Comando enviado:
CONFIG 0 2 0.000
Confirmacion recibida desde C:
planificador=0, memoria=2
Resultado:
La politica puede cambiar sin reiniciar el ejecutable ni recrear los PCB. El cambio se aplica a las asignaciones futuras; no recoloca bloques existentes.

3.5 Insercion de procesos durante ejecucion
Mientras P1 se encontraba ejecutandose, se envio:
ADD P2 64 2.000 0.000 0
Resultado:
P2 fue recibido por C y aparecio en los snapshots sin reiniciar la simulacion. Durante ejecucion se envia solamente ADD, no se reenvia CONFIG ni SPEED.

3.6 Error por memoria invalida
Se ingreso un proceso de 8192 KB con una memoria fisica simulada de 4096 KB.
Resultado:
state: ERROR
code: ERR_MEM
description: Memoria invalida: 8192 KB

3.7 Generacion aleatoria
El comando RANDOM genera actualmente cinco procesos.
Resultado observado:
created_processes.count = 5
Limitacion:
La cantidad no puede ser elegida por el usuario; es una constante.

3.8 Sanitizadores
Se ejecuto un caso Round Robin + Best Fit con AddressSanitizer y UndefinedBehaviorSanitizer.
Resultado:
No se detectaron accesos invalidos ni comportamiento indefinido.
Nota:
La deteccion de fugas de memoria de AddressSanitizer no esta soportada por esta plataforma. Las fugas pendientes se identificaron por inspeccion.

4. ARQUITECTURA ACTUAL

4.1 Capa C

Archivo:

src/main.c

Responsabilidades: - Estado global del simulador. - PCB.

- CPU y cambio de contexto. - Colas de procesos. - Planificadores.

- Asignacion de memoria. - Entrada/salida.

- Gantt.

- Protocolo de comandos.

- Serializacion JSON para Python.

Inventario:

- 6 enum.

- 14 struct principales. - 59 definiciones de funciones C.

Limitacion arquitectonica: Todas las responsabilidades siguen concentradas en un solo archivo de mas de 1500 lineas.

4.2 Capa Python

Entrada:

```

src/main.py
Puente:
src/gui/simulation_client.py
Controlador comun:
src/gui/algorithm_window.py
Interfaz:
src/gui/main_window.py src/gui/*_window.py src/gui/components/*.py
Inventario aproximado: - 22 clases Python. - 151 funciones y metodos, incluyendo func
iones internas.
4.3 Protocolo de comunicacion
Python -> C:
CONFIG <scheduler> <memory> <quantum> ADD <name> <memory_kb> <burst> <arrival> <prior
ity> RANDOM
SPEED <value>
RUN
PAUSE
STOP
C -> Python:
SIM_DATA snapshot
SIM_DATA queue SIM_DATA running SIM_DATA memory SIM_DATA gantt
Cada snapshot permite reconstruir la simulacion completa en la GUI.
-----

```

5. FLUJO ACTUAL DE UN PROCESO

```

ADD / RANDOM
|
v
created_processes Estado NEW
Espera arrival_time
|
| process_arrival()
v
job_q Proceso llegado, pendiente de memoria |
| job_scheduler() | kmalloc()
v
ready_q Estado READY
|
| scheduler()
v
next_pcb
|
| dispatch() | context_switch()
| PCB.cpu_ctx -> Simulator.cpu_ctx
v
running Estado RUNNING
|
+-----+-----+-----+-----+
| | |
| burst terminado | quantum/apropiacion | inicio de E/S
v v v
dispatch_save_ctx() dispatch_save_ctx() dispatch_save_ctx()
| | |
v v v
finished_q ready_q io_q TERMINATED READY BLOCKED
|
| tick_io_q() | blocked_to_ready()
v
ready_q
-----

```

6. COLAS Y ESTRUCTURAS DE CONTROL

6.1 created_processes

```

Tipo:
struct Queue
Contenido:
PCB en NEW cuyo arrival_time aun no se cumple.
Productores:
- process_stdin() mediante ADD. - create_random_processes() mediante RANDOM. - Inserc
ion manual desde la GUI durante ejecucion.
Consumidor:
- process_arrival().
Destino:
- job_q.
Estado:
[OK - PROBADO]
6.2 job_q
Contenido:
Procesos que ya llegaron, pero esperan asignacion de memoria.

```

Productor:
 - process_arrival().
 Consumidor:
 - job_scheduler().
 Resultados:
 - ready_q si kmalloc() tiene exito. - finished_q con ERROR si la memoria solicitada es invalida. - permanece en job_q si no existe un hueco suficientemente grande.
 Estado:
 [OK - PROBADO]
 6.3 ready_q
 Contenido:
 Procesos READY residentes en memoria.
 Productores:
 - job_scheduler(). - Fin de quantum. - Apropiacion. - blocked_to_ready().
 Consumidores:
 - alg_fcfs(). - alg_sjf(). - alg_priority().
 Estado:
 [OK - PROBADO]
 6.4 io_q
 Contenido:
 Procesos BLOCKED esperando que finalice una operacion de E/S.
 Productor:
 - running_to_blocked().
 Consumidor:
 - tick_io_q().
 Salida:
 - blocked_to_ready() mueve el PCB a ready_q.
 Estado:
 [OK - PROBADO] para una operacion de E/S por proceso.
 Limitaciones:
 - Cola unica para los tres dispositivos. - No existe prioridad ni controlador por dispositivo. - Un proceso solo puede realizar una operacion de E/S.
 6.5 finished_q
 Contenido:
 Procesos TERMINATED y ERROR.
 Productores:
 - terminate_running_process(). - job_scheduler() para ERR_MEM.
 Estado:
 [OK - PROBADO]
 6.6 running
 Tipo:
 struct Pcb *
 Representa el proceso que tiene la CPU.
 No es una cola.
 Estado:
 [OK - PROBADO]
 6.7 next_pcb
 Proceso seleccionado por scheduler() antes del despacho.
 Estado:
 [OK - CODIGO]

7. ENUMERACIONES

7.1 SimulatorState

SIM_RUN
 SIM_PAUSE
 SIM_STOP

Estado:
 [OK - PROBADO]

7.2 AlgorithmSched

FCFS
 NP_SJF
 P_SJF
 ROUND
 NP_PRIOR
 P_PRIOR

Estado:
 [OK - PROBADO]

7.3 AlgorithmMem

FIRST
 BEST
 WORST

Estado:
 [OK - PROBADO]

7.4 ProcessState

NEW
 READY
 RUNNING

BLOCKED
TERMINATED
ERROR_STATE
Estado:
[OK - PROBADO]
7.5 IoDevice
IO_NONE
IO_KEYBOARD
IO_DISK
IO_PRINTER

Estado:
[OK - PROBADO] como seleccion aleatoria y etiqueta. [PARCIAL] como sistema de dispositivos independientes.

7.6 IntType
INT_HW_IO
INT_HW_TIMER
INT_SW_SYSCALL
INT_EXC_DIV_ZERO
INT_EXC_MEM

Estado:
[PENDIENTE]
Los valores estan definidos, pero no son generados ni atendidos.

8. STRUCT Y SEGUIMIENTO DE FUNCIONES

8.1 struct CpuContext

Campos:

- program_counter - stack_pointer

Seguimiento:

create_pcb() -> inicializa el contexto guardado del proceso.

simulator_init()

-> inicializa el contexto activo de la CPU.

dispatch_restore_ctx() -> copia Pcb.cpu_ctx hacia Simulator.cpu_ctx.

tick_running_process() -> incrementa Simulator.cpu_ctx.program_counter.

dispatch_save_ctx() -> copia Simulator.cpu_ctx hacia Pcb.cpu_ctx.

context_switch()

-> coordina guardado y restauracion.

Estado:

[OK - PROBADO] para program_counter. [PARCIAL] stack_pointer existe, pero nunca cambia.

Observacion:

send_pcb_data() envia Pcb.cpu_ctx. Mientras el proceso esta ejecutandose, el valor mas reciente se encuentra en Simulator.cpu_ctx. Por ello, el PC mostrado en vivo para running puede quedar atrasado hasta el siguiente guardado.

8.2 struct SchedulerData

Campos: arrival_time, burst_time, remaining_time, start_time, finish_time, waiting_time, turnaround_time, response_time, priority, remaining_quantum.

Funciones:

create_pcb() dispatch() tick_running_process() terminate_running_process() alg_sjf()

alg_priority() should_preempt()

Estado:

[OK - PROBADO]

Precision pendiente: $\text{waiting_time} = \text{turnaround_time} - \text{burst_time}$. Cuando existe E/S, esta formula tambien contabiliza la espera de E/S como espera de ready_q.

8.3 struct MemoryData

Campos: required_kb, assigned_blocks, waste_kb, start, limit.

Funciones:

create_pcb() allocate_block()

kmalloc()

kfree()

send_pcb_data()

Estado:

[OK - PROBADO]

8.4 struct IoData

Campos: has_io, started, completed, start_time, duration, remaining_time, device.

Funciones:

create_pcb() tick_running_process() running_to_blocked() tick_io_q() blocked_to_ready()

Estado:

[OK - PROBADO] para una operacion aleatoria.

Pendiente:

Multiples solicitudes, codigos de interrupcion, controladores y colas por dispositivo

8.5 struct Interrupt

Campos: int_count, int_done, next_int.

Funciones:

create_pcb() calcula los valores. send_pcb_data() los envia a Python.

Estado:

[PARCIAL]

int_done no aumenta y next_int no dispara ninguna accion.

8.6 struct ErrorData

Campos: has_error, error_code, error_desc.

Funciones:

create_pcb() job_scheduler() send_pcb_data()

Estado:

[PARCIAL]

ERR_MEM funciona. El error aleatorio se genera, pero no existe un modulo que lo detecte durante ejecucion y cancele el proceso.

8.7 struct Pcb

Agrupar: - Identidad.

- Estado.

- Contexto CPU.

- Datos de memoria.

- Datos de planificacion.

- E/S.

- Error.

- Interrupciones.

Funciones principales: create_pcb() create_random_processes()

send_pcb_data() print_pcb()

Estado:

[OK - PROBADO] como estructura central.

Pendiente:

set_process_state() esta declarada, pero no implementada.

8.8 struct Node y struct Queue

Queue: cont, head, tail.

Funciones:

init_queue() enqueue() dequeue_last() dequeue_head() dequeue_tail() dequeue_pcb() q_e

empty() q_free()

Estado:

[OK - PROBADO]

Deuda tecnica:

dequeue_*() y dequeue_pcb() asumen que la cola y el proceso son validos. No tienen proteccion completa ante una cola vacia o un PCB inexistente.

8.9 struct MemoryBlock y struct MemoryBlockList

MemoryBlock: start, limit, length, owner, next.

MemoryBlockList: cont, head, tail, max, free.

Funciones:

mem_init()

allocate_block()

kmalloc()

kfree()

kmerge() update_max_mem() send_memory_data()

Estado:

[OK - PROBADO]

8.10 struct GanttNode y struct GanttList

Funciones:

gantt_init() update_gantt_tick() send_gantt_nodes() send_gantt_data()

Estado:

[OK - PROBADO]

El diagrama registra segmentos PROCESS, IDLE y CONTEXT_SWITCH. Los segmentos consecutivos equivalentes se fusionan y cada cambio de contexto consume un TICK.

8.11 struct Simulator

Agrupar: - Estado global. - Algoritmos seleccionados. - Cinco colas.

- Memoria.

- CPU y contexto activo. - PCB running y next_pcb. - Gantt.

- Tiempo, quantum, velocidad y PID.

Funciones:

simulator_init()

process_stdin() main_loop() send_data()

main()

Estado:

[OK - PROBADO]

9. MODULOS CONCEPTUALES DEL NUCLEO C

9.1 Inicializacion

init_queue() mem_init()

gantt_init() simulator_init()

Estado:

[OK - PROBADO]

Deuda:

Permanece declarada ctx_init(), aunque ContextList fue eliminado.

9.2 Gestion de colas

enqueue() dequeue_last() dequeue_head() dequeue_tail() dequeue_pcb() q_empty() q_free()

Estado:

[OK - PROBADO]

9.3 Memoria

allocate_block()

kmalloc()

kfree()

kmerge() update_max_mem()

Estado:

[OK - PROBADO]

Politica dinamica:

La GUI permite cambiar First/Best/Worst Fit durante ejecucion. El nuevo valor se envia mediante CONFIG y afecta las proximas asignaciones.

9.4 Planificador de largo plazo

process_arrival() job_scheduler()

Estado:

[OK - PROBADO]

9.5 Planificador de corto plazo

scheduler()

alg_fcfs() alg_sjf()

alg_priority() should_preempt()

Estado:

[OK - PROBADO]

9.6 Planificador de mediano plazo

Estado:

[PENDIENTE]

No existen estados suspendidos, swapping ni suspension temporal.

9.7 Dispatcher

dispatch() dispatch_save_ctx() dispatch_restore_ctx() context_switch()

Estado:

[OK - PROBADO]

El modelo actual diferencia:

Simulator.cpu_ctx = registros activos de la CPU. Pcb.cpu_ctx = registros guardados de 1 proceso.

9.8 Ejecucion de CPU

tick_running_process() terminate_running_process()

Estado:

[OK - PROBADO]

9.9 Entrada/salida

running_to_blocked() tick_io_q() blocked_to_ready()

Estado:

[OK - PROBADO] para el ciclo basico. [PARCIAL] respecto al requerimiento completo de dispositivos.

9.10 Interrupciones y señales

Elementos existentes:

enum IntType

struct Interrupt Simulator.interrupt_q

Estado:

[PENDIENTE]

interrupt_q es un unico enum, no una cola. No hay ISR, prioridades, duracion, generacion, atencion ni retorno de interrupcion.

9.11 Errores

Elementos:

ERROR_STATE

ErrorData

ERR_MEM

Probabilidad aleatoria de 0.5%

Estado:

[PARCIAL]

9.12 Gantt y estadisticas

update_gantt_tick() send_gantt_data() _build_stats()

Estado:

[OK - PROBADO] para Gantt y utilizacion de CPU por tiempo PROCESS.

[PARCIAL] para waiting_time cuando existe E/S.

9.13 Protocolo C/Python

stdin_has_data()

send_running_data() process_stdin() send_json_string() send_memory_data() send_gantt_

nodes() send_queue_data() send_gantt_data() send_pcb_data() send_data()

Estado:

[OK - PROBADO]

Deuda:
sscanf() no valida la cantidad de campos recibidos. SPEED no valida cero o valores negativos. CONFIG no valida rangos de enum.

9.14 Debug y log

Logs: log_pause() log_config() log_run() log_add() log_stop()

Debug: print_pcb() print_queue() print_memory() print_main_loop()

Estado:

[OK - CODIGO]

print_main_loop() es una salida legacy reemplazada por send_data().

10. MODULOS PYTHON

10.1 src/main.py

- Inicia QApplication. - Aplica paleta. - Crea SimulationClient.
- Muestra MainWindow.

Estado:

[OK - PROBADO]

10.2 simulation_client.py

Clases:

ProcessData

UiProcess

SimulationResult

SimulationClient

Responsabilidades: - Iniciar el ejecutable C. - Enviar comandos.

- Leer stdout/stderr con threads.

- Parsear JSON.

- Mantener estado por algoritmo. - Reconstruir UiProcess.

- Construir mapa de memoria y estadísticas. - Cambiar política de memoria dinámicamente. - Agregar procesos durante ejecución enviando solo ADD.

Estado:

[OK - PROBADO]

Deuda:

El parametro apply_events de run() no se utiliza. SimulationResult.returncode y algunos caminos de events/stdout_lines pertenecen a una interfaz anterior o alternativa.

10.3 algorithm_window.py

Responsabilidades: - Control común de las seis ventanas.

- Agregar procesos. - Agregar cinco procesos aleatorios. - Ejecutar, pausar, continuar y detener. - Cambiar velocidad.

- Cambiar memoria durante ejecución. - Actualizar interfaz cada 120 ms.

Estado:

[OK - PROBADO]

10.4 main_window.py

Responsabilidad: Menu de selección de algoritmo de CPU.

Estado:

[OK - PROBADO]

10.5 Ventanas de algoritmos

fcfs_window.py pra_window.py prn_window.py

sjfa_window.py sjfn_window.py rr_window.py Todas heredan de AlgorithmWindow.

Estado:

[OK - PROBADO]

10.6 Componentes

process_input.py Formulario y controles.

process_queue.py Tarjetas visuales de procesos.

execute_tab.py CPU, Gantt, memoria, contadores, estadísticas, PCB y log.

visual_widgets.py GanttWidget, MemoryMapWidget, StateChip y GlowLabel.

header.py Estado, tiempo y selector dinámico de memoria.

footer.py CPU, procesos, terminados, memoria y políticas.

center.py Distribución principal.

theme.py Estilos PySide6.

Estado:

[OK - PROBADO]

11. COMPARACION CON LOS REQUERIMIENTOS HISTORICOS

11.1 Ingreso manual de procesos

Requerimiento: Ingresar procesos con memoria y burst-time.

Estado actual:

[OK - PROBADO]

Adicional:

Se pueden agregar procesos durante la ejecución.

11.2 Procesos generados por el sistema

Requerimiento: Generación aleatoria, indicando cantidad.

Estado actual:

[PARCIAL]

Existe RANDOM, pero siempre genera cinco procesos.

11.3 Cinco estados

Requerimiento:

NEW, READY, RUNNING, BLOCKED y TERMINATED.

Estado actual:

[OK - PROBADO]

Adicional:

ERROR_STATE.

11.4 PCB y Program Counter

Requerimiento: PCB con campos relevantes y uso del PC.

Estado actual:

[OK - PROBADO] para estructura y cambio de contexto. [PARCIAL] para visualización del PC activo y stack pointer.

11.5 Colas de procesos

Requerimiento: Procesos totales, listos y E/S.

Estado actual:

[OK - PROBADO]

Se implementan: created_processes, job_q, ready_q, io_q y finished_q.

11.6 Planificadores

Requerimiento: FCFS, SJF, Round Robin y prioridades en contextos apropiativos y no apropiativos.

Estado actual:

[OK - PROBADO]

Existen seis modos seleccionables desde la GUI.

11.7 Quantum configurable

Requerimiento: Ingresar y volver a cambiar el quantum.

Estado actual:

[PARCIAL]

Puede ingresarse antes de Round Robin. La GUI no proporciona actualmente un evento específico para cambiarlo durante la ejecución.

11.8 Dispatcher y cambio de contexto

Requerimiento: Guardar y restaurar el contexto.

Estado actual:

[OK - PROBADO]

11.9 Errores del proceso

Requerimiento: Cancelar procesos incorrectos y almacenar código en PCB/estadísticas.

Estado actual:

[PARCIAL]

ERR_MEM funciona. El error aleatorio de 0.5% solo se etiqueta.

11.10 Interrupciones entre 5 y 20

Requerimiento: Generar y atender interrupciones aleatorias.

Estado actual:

[PARCIAL]

La cantidad se calcula entre 5 y 20, pero no existen interrupciones reales.

11.11 Duración de interrupciones

Requerimiento: Duración aleatoria entre 5 y 20 unidades.

Estado actual:

[PENDIENTE]

La E/S tiene duración aleatoria entre 1 y 10, pero no se modela una duración general por interrupción.

11.12 Actividad en línea

Requerimiento:

Mostrar estados, PCB, PC y otros datos relevantes.

Estado actual:

[OK - PROBADO]

La GUI muestra:

- Estado.
- Progreso. - PCB.
- Memoria.
- Gantt.
- Estadísticas.
- Logs.

11.13 Veinte procesos y comparación

Requerimiento: Ejecutar 20 procesos y comparar políticas.

Estado actual:

[PARCIAL]

El sistema admite 20 o más procesos, incluso de forma secuencial. No existe una batería automatizada ni un informe comparativo almacenado.

11.14 Proceso cargado completamente en memoria

Requerimiento: El proceso debe cargarse completamente antes de iniciar.

Estado actual:

[OK - PROBADO]

Un PCB no pasa a ready_q hasta que kmallocc() tenga éxito.

11.15 Stack, heap y memoria variable

Requerimiento: Asignación dinámica stack/heap durante ejecución.

Estado actual:

[PENDIENTE]

Solo existe una asignación total al admitir el proceso.

11.16 Direccionamiento físico

Requerimiento: Dirección base en PCB sin MMU.

Estado actual:

[OK - PROBADO]

Pcb.mem.start y Pcb.mem.limit se muestran como base y límite.

11.17 Lista enlazada o mapa de bits

Requerimiento: Elegir una estructura de administración.

Estado actual:

[OK - PROBADO]

Se utiliza MemoryBlockList enlazada.

11.18 Tres estrategias de asignación

Requerimiento: First Fit, Best Fit y Worst Fit.

Estado actual:

[OK - PROBADO]

Adicional:

Se pueden cambiar durante la ejecucion para nuevas asignaciones.

11.19 Segmentacion en bloques y desperdicio

Requerimiento: Bloques potencia de dos y control de desperdicio.

Estado actual:

[PARCIAL]

La memoria tiene 1024 bloques de 4 KB. El proceso se redondea al numero de bloques requerido y se calcula waste_kb. No se modelan por separado programa, datos y memoria variable.

11.20 Dispositivos de E/S

Requerimiento: Teclado, disco e impresora.

Estado actual:

[OK - PROBADO] para seleccion y espera basica. [PARCIAL] para administracion completa por dispositivo.

11.21 Interrupcion de solicitud y finalizacion de E/S

Requerimiento: Generar codigos de interrupcion al iniciar y terminar E/S.

Estado actual:

[PENDIENTE]

El cambio de estado funciona directamente, sin una interrupcion formal.

11.22 Colas por controlador

Requerimiento: Administrar las colas de atencion de dispositivos.

Estado actual:

[PARCIAL]

Existe una unica io_q compartida.

11.23 Teclado: cancelacion o continuacion

Estado actual:

[PENDIENTE]

11.24 Procesos del SO y procesos de usuario

Estado actual:

[PENDIENTE]

No existe un campo que identifique el tipo de proceso.

11.25 Informe, instalacion y manual de usuario

Estado actual:

[PARCIAL]

Este documento actualiza la documentacion tecnica. Aun faltan:

- Manual de instalacion formal.
- Manual de usuario.
- Requisitos para Windows. - Presentacion final.
- Conclusiones experimentales con 20 procesos.

12. FUNCIONALIDAD COMPLETADA Y FUNCIONANDO

Lista consolidada:

[OK] Creacion manual de procesos. [OK] Creacion aleatoria de cinco procesos. [OK] Insercion de procesos durante ejecucion. [OK] Estados NEW, READY, RUNNING, BLOCKED, TERMINATED y ERROR. [OK] Colas created, job, ready, io y finished. [OK] FCFS.

[OK] SJF no apropiativo. [OK] SJF apropiativo. [OK] Round Robin.

[OK] Prioridad no apropiativa. [OK] Prioridad apropiativa. [OK] Quantum inicial de Round Robin.

[OK] Apropiacion SJF y prioridad. [OK] Contexto CPU separado del contexto guardado de PCB. [OK] Guardado/restauracion de program counter. [OK] First Fit.

[OK] Best Fit.

[OK] Worst Fit.

[OK] Cambio dinamico de politica de memoria. [OK] Asignacion total antes de READY. [OK] Direcciones base y limite. [OK] Calculo de bloques y desperdicio interno. [OK] Liberacion de memoria.

[OK] Fusion de huecos libres.

[OK] Espera en job_q cuando no existe hueco suficiente. [OK] Error por memoria invalida. [OK] E/S basica y bloqueo. [OK] Retorno de io_q a ready_q. [OK] Teclado, disco e impresora como tipos de dispositivo. [OK] Pausa, continuacion, detencion y velocidad.

[OK] Snapshots JSON. [OK] Mapa visual de memoria. [OK] Tabla PCB.

[OK] Gantt con segmentos PROCESS, IDLE y CONTEXT_SWITCH. [OK] Estadisticas basicas.

[OK] Log de intercambio GUI/C.

13. FUNCIONALIDAD A MEDIAS

[PARCIAL] Interrupciones: Datos creados, comportamiento ausente.

[PARCIAL] Errores:

ERR_MEM funciona; error aleatorio no se ejecuta.

[PARCIAL] E/S:

Solo una operacion por proceso y una cola compartida.

[PARCIAL] Dispositivos: Tres tipos definidos, sin controlador propio.

[PARCIAL] Contexto:

PC funcional; stack pointer no cambia.

[PARCIAL] PC en vivo:

La serializacion usa la copia guardada del PCB.

[PARCIAL] Espera: Puede incluir tiempo bloqueado por E/S.

[PARCIAL] Quantum: Configurable al inicio, no mediante un selector dinamico dedicado.

[PARCIAL] Generacion aleatoria:

Cantidad fija en cinco.

[PARCIAL] Comparacion de politicas: Algoritmos disponibles, estudio formal ausente.

[PARCIAL] Documentacion:

Informe tecnico presente; faltan manuales.

14. FUNCIONALIDAD NO IMPLEMENTADA

[PENDIENTE] Cola real de interrupciones. [PENDIENTE] ISR o manejadores de interrupcion. [PENDIENTE] Interrupciones TIMER. [PENDIENTE] Syscalls simuladas. [PENDIENTE] Excepcion de division por cero. [PENDIENTE] Excepcion de acceso a memoria durante ejecucion. [PENDIENTE] Duracion individual de interrupciones.

[PENDIENTE] Diez interrupciones de E/S por proceso. [PENDIENTE] Controlador y cola por dispositivo. [PENDIENTE] Captura de señal de teclado. [PENDIENTE] Cancelacion/continuacion por teclado. [PENDIENTE] Planificador de mediano plazo. [PENDIENTE] Procesos suspendidos. [PENDIENTE] Swapping. [PENDIENTE] Stack y heap. [PENDIENTE] Asignacion variable durante ejecucion. [PENDIENTE] Liberacion parcial de memoria. [PENDIENTE] Segmentos separados de codigo y datos. [PENDIENTE] Procesos del SO diferenciados de usuarios.

[PENDIENTE] Cantidad configurable de procesos aleatorios. [PENDIENTE] Estudio automatizado de 20 procesos. [PENDIENTE] Manual de instalacion.

[PENDIENTE] Manual de usuario.

[PENDIENTE] Soporte Windows nativo.

15. DEUDAS TECNICAS Y CORRECCIONES RECOMENDADAS

Prioridad alta:

1. Corregir el PC mostrado para el proceso running. send_running_data() debe serializar Simulator.cpu_ctx o sincronizar la copia antes de enviar.

2. Agregar una metrica separada para el overhead de cambios de contexto.

3. Separar waiting_time de blocked_time. Agregar acumuladores ready_wait_time e io_wait_time.

4. Implementar una cola real de interrupciones.

5. Hacer efectivo el error aleatorio.

Prioridad media:

6. Eliminar la declaracion obsoleta ctx_init().

7. Implementar o eliminar set_process_state().

8. Validar sscanf() y rangos de CONFIG, ADD y SPEED.

9. Proteger dequeue_*() y dequeue_pcb().

10. Permitir cambio dinamico de quantum.

11. Permitir elegir la cantidad de procesos RANDOM.

12. Agregar metricas de fragmentacion externa y total de desperdicio.

13. Liberar los nodos Gantt y MemoryBlockList al terminar.

Prioridad de arquitectura:

14. Dividir src/main.c en archivos .c/.h.

15. Agregar pruebas automatizadas permanentes.

16. Agregar Makefile o sistema de build.

17. Excluir build/, dSYM y ejecutables del control de versiones.

18. Documentar Linux/macOS/Windows.

16. DIVISION RECOMENDADA DEL CODIGO C

include/

simulator.h

pcb.h queue.h memory.h scheduler.h

dispatcher.h io.h

interrupt.h gantt.h protocol.h

src/

simulator.c

pcb.c

queue.c

memory.c scheduler.c

dispatcher.c io.c

interrupt.c

gantt.c protocol.c main.c

Correspondencia:

queue.c init_queue, enqueue, dequeue_*, q_empty, q_free.

pcb.c create_pcb, create_random_processes, set_process_state.

memory.c mem_init, allocate_block, kmalloc, kfree, kmerge, update_max_mem.

scheduler.c

process_arrival, job_scheduler, scheduler, alg_*, should_preempt.

dispatcher.c dispatch, dispatch_save_ctx, dispatch_restore_ctx, context_switch.

io.c

running_to_blocked, blocked_to_ready, tick_io_q.

interrupt.c Cola, generacion, handlers y retorno de interrupcion.

gantt.c gantt_init, update_gantt_tick, send_gantt_data.

protocol.c process_stdin y funciones send_*.

simulator.c

simulator_init, tick_running_process, main_loop.

17. PLAN DE TRABAJO SUGERIDO

Fase 1 - Precision del simulador:

- ☐ PC activo correcto en GUI.
- ☒ Gantt sin perdida de ticks, con IDLE y cambios de contexto. ☐ Tiempos READY y BLOCKED separados. ☐ Pruebas unitarias de colas y memoria.

Fase 2 - Interrupciones:

- ☐ struct InterruptEvent. ☐ Queue interrupt_q. ☐ Generador TIMER e I/O.
- ☐ ISR.
- ☐ Duracion y prioridad. ☐ Estadisticas de interrupciones.

Fase 3 - E/S avanzada:

- ☐ Cola por teclado. ☐ Cola por disco. ☐ Cola por impresora. ☐ Multiples operaciones por proceso. ☐ Señal de teclado.

Fase 4 - Memoria avanzada:

- ☐ Stack.
- ☐ Heap. ☐ Asignacion/liberacion parcial. ☐ Fragmentacion externa. ☐ Comparacion First/Best/Worst Fit.

Fase 5 - Entrega:

- ☐ Manual de instalacion.
- ☐ Manual de usuario.
- ☐ Script de compilacion. ☐ Compatibilidad Windows. ☐ Escenario de 20 procesos.
- ☐ Graficos comparativos. ☐ Presentacion final.

18. CONCLUSION

El proyecto ya supero la etapa de prototipo visual. El nucleo actual ejecuta procesos, cambia contextos, administra memoria, bloquea por E/S y comunica su estado a una interfaz grafica en tiempo real.

Los componentes mas importantes del flujo base estan implementados y fueron probados:

- Seis modos de planificacion. - Tres politicas de memoria. - Cinco colas de procesos

- Contexto CPU/PCB.
- E/S basica.
- Gantt y estadisticas. - Configuracion dinamica de memoria. - Insercion de procesos en caliente.

La principal distancia respecto a los requerimientos historicos se concentra en interrupciones, señales, E/S avanzada, stack/heap, planificacion de mediano plazo y experimentacion formal con 20 procesos.

La prioridad recomendada es corregir la precision del PC y tiempos de espera, y despues construir el modulo de interrupciones. Con esos cambios, el simulador tendria una correspondencia mucho mas completa con los temas de Sistemas Operativos planteados en el documento de referencia.

FIN DEL INFORME